CS 188 Final Project Report

Sanjay Dhamodharan and Aneesh Ratnala

Problem Statement

For our final project, we created a robot arm that is **fully voice-controlled**, integrating Robosuite with the Vosk API. Furthermore, to benchmark the capabilities of a voice-controlled robot, we attempt two difficult tasks in Robosuite: **nut assembly** and **multi-cube stacking**. To achieve this, we present several commands that can be used to dexterously control the robot. These include low-level position commands such as "forward," "back," "up," and "down," and high-level waypoint-based commands such as "grab," "hover," and "place." Our highest-level voice commands, like "stack" and "assemble," can execute the entirety of a given task. This idea was inspired by the sample problem statement (**multimodal processing**) found in the project rubric.

Methods

Our pipeline has three stages:

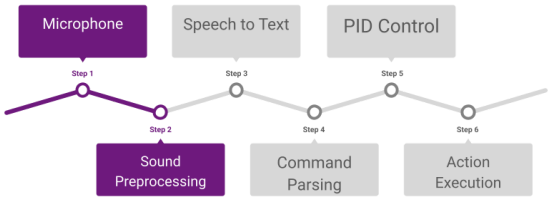
1. Speech recognition,
2. Keyword-to-mode mapping, and
3. Motion generation, which differs for incremental jog commands versus waypoint-based stack/nut policies.

Speech recognition

We run Vosk locally on the CPU using a small English language model (vosk-model-small-en-us-0.15). Audio is captured with SoundDevice, read in ~0.25 second blocks at the device's native rate. Each block is downmixed to mono, resampled to 16 kHz with polyphase resampling, peak-normalized when samples exceed ±1 in float, then converted to 16-bit PCM with clipping to the valid integer range. Chunks are passed to Vosk's **KaldiRecognizer**, a core class in their speech recognition library. We call SetGrammar with a JSON list of *allowed command strings* so decoding is restricted to our constrained vocabulary, which improves reliability for short keywords compared to open-ended dictation.

Command routing

Recognized text is matched against keyword sets that depend on **context** (e.g. wait for start, then stack/grab/assemble/incremental jog words, or hover/place in segmented motion). This determines which controller runs and whether the user is in incremental teleop, segmented grasp–hover–place, or a full autonomous stack/assemble sequence.



Motion generation

We have three main types of motion:

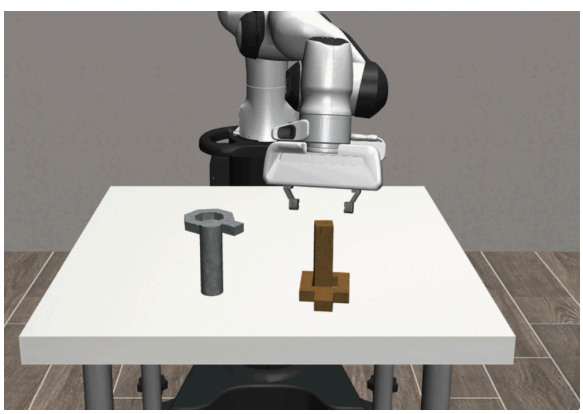
1. **Fully autonomous** motion ("stack" and "assemble"),
2. **Segmented** autonomous motion (grab-hover-place workflow), and
3. **Incremental** control ("up," "down," "left," "right," "forward," "back," "clockwise," and "counterclockwise").

Incremental teleop

Commands such as up, down, left, right, forward, back, clockwise, and counterclockwise do not set a Cartesian target relative to the current end-effector. They apply a fixed delta in world axes (e.g. +Z for "up") of 0.048 m per control step, clipped to ±0.14 m per axis, repeated for 26 steps at the environment control rate (20 Hz in our scripts). Each spoken jog therefore produces a **short burst of motion**, intentionally discrete to stay usable under speech latency.

Waypoint-based stack and nut tasks

For stack/assemble and the grab-hover-place workflow, policies use observations cubeA_pos and cubeB_pos as "grasp object" and "place/support object." For multi-layer stacking, we assign those observation fields each layer to the appropriate object pair. A PID controller tracks named waypoints. Segmented runs, as opposed to the fully autonomous runs, **pause** between the grab, hover, and place phases until the next keyword, using the same geometric targets as the fully autonomous run.



Waypoint-based, voice-induced grab-hover-place workflow in the nuts & pegs environment

Evaluation

To evaluate, we had **six** main success criteria based on our three motion styles for each of our two environments. Our goal was as follows:

1. In the multi-cube stacking environment, using each of our three voice-controlled motion styles, stack at least 3 cubes at a greater than 80% rate, and
2. In the nuts and pegs environment, using each of our three voice-controlled motion styles, successfully place each nut in its peg at a greater than 80% rate.

Percent of successes (out of 10 trials per case) given motion style and environment

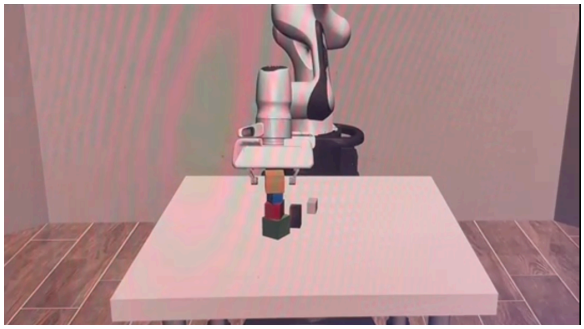
	Multi-cube	Nuts & pegs
Fully autonomous	100%	100%
Segmented	100%	100%
Incremental	90%	80%

Across the two tasks, our **fully autonomous** and **segmented** autonomous modes benefited from the same underlying waypoint-style manipulation approach that we built earlier for Assignment 1. These modes follow a structured sequence of phases (e.g., hover, grasp, hover, place), which makes them inherently easier to tune and stabilize.

In contrast, **incremental** control was the most challenging mode. Because speech recognition introduces latency and because our incremental commands operate in a bursty manner, the robot does not continuously stream motion toward a target. Instead, each jog word generates a short, fixed burst of motion in predefined world-frame directions before stopping. This means the user must correct and re-command repeatedly, and

small recognition/control timing differences can noticeably affect whether the end-effector stays well-aligned during contact-heavy parts of the task (especially in nuts and pegs). Overall, this different "burst teleop" nature, rather than the basic grasp-and-place mechanics, was what made incremental performance the hardest to achieve consistently. However, given the consistent nature of the voice detection and the robot's movements, we still successfully stacked three cubes 9 out of 10 times and fit nuts in their pegs 8 out of 10 times—just with a little more effort.

Ultimately, we **met** our 80% threshold for all six of our success criteria.



Our robot in the multi-stacking cubes environment

Discussion

Challenges

The most difficult part of this project was the integration and latency of the Vosk voice-to-text model. Our initial plan was to have a robot that moves continuously, only changing plans when the user issues a voice command; however, due to the model's latency, the model would change course about 2 seconds after a command's issuing. This was a dealbreaker in an environment like ours that requires precise movement. We predict that if we use a much stronger GPU for inference, we can reduce the latency drastically, hopefully enabling continuous action as opposed to discrete steps.

Further, we reflected on some challenges that came with implementing an incremental teleop policy. Because the user provides control in short spoken "step" commands, the system must translate each recognized word into a small motion while also handling misrecognitions and timing gaps between utterances. In practice, this meant carefully tuning step size and clamping targets to keep motion precise and avoid cumulative drift over repeated commands. We also had to ensure the robot behaved predictably when commands arrived late or out of order (for example, by limiting continuous drift and maintaining a consistent stop-and-wait cadence).

Future development

Currently, our robot only has 4 of the 6 degrees of freedom, lacking roll and pitch, as we used Assignment 1's robot & environments as a base for our project. Increasing the degrees of freedom could open up to even harder tasks, like opening a door.

For future directions, a promising approach would be to attach a vision-language model between the voice-to-text model and position planning. This would enable more complex natural language instructions for the robot arm.

Team Contributions

In this project, AI was used to assist creating our website, the functions for low-level commands, test case generation, and the implementation of our incremental tele-op policy, with heavy oversight. Aneesh handled the voice integration, PID, and evaluation while Sanjay worked on the high- and low-level position control, website, and mapping text to actions. We also worked

together ensure testing scripts worked as intended, and we made sure that all files were "polished" at the end.